



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.03 Прикладная информатика

О Т Ч Е Т

по индивидуальному заданию

Название: Разработка микросервиса экспорта данных

Дисциплина: Проектирование информационных систем

Студент

ИУ6-75Б

(Группа)

(Подпись, дата)

Н.А.Николаева

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

А.Ф. Прутик

(И.О. Фамилия)

Москва, 2025

Цель работы: Разработать автономный микросервис для системы "Parser", обеспечивающий просмотра результатов парсинга веб-сайтов.

Требования к системе:

Система должна обеспечивать хранение и предобработка структурированных результатов завершенной парсинг-сессии с удалением дубликатов и нормализацией цен и единиц, с возможностью фильтрации по наличию, цене и другим характеристикам. Сервис должен предоставлять REST API для интеграции с другими компонентами системы и веб-интерфейс для конечных пользователей.

Система должна работать независимо от других микросервисов, обеспечивая время отклика API менее 2 секунд и поддерживая экспорт до 10000 записей за один запрос.

1 Описание решения и используемые технологии:

Для реализации выбрана микросервисная архитектура с использованием контейнеризации. В качестве основного фреймворка бэкенда используется FastAPI на Python 3.11, выбранный за высокую производительность благодаря асинхронности, автоматическую генерацию документации в форматах Swagger и OpenAPI. База данных PostgreSQL версии 16 обеспечивает надежное хранение данных парсинга и истории экспортов. Для взаимодействия с базой данных применяется ORM SQLAlchemy.

Фронтенд системы взаимодействует с микросервисом через REST API и обеспечивает доступ к данным о результатах парсинга. Пользовательский интерфейс формирует HTTP-запросы к backend-сервису и получает ответы в формате JSON. Основная бизнес-логика реализована на стороне серверного приложения.

Серверная часть разработана на языке Python с использованием фреймворка FastAPI, который обеспечивает высокую производительность и удобную реализацию REST API. Для хранения данных используется система управления базами данных PostgreSQL. Вся инфраструктура упакована в Docker контейнеры и управляется через Docker Compose.

2 Архитектурные схемы

На рисунке 1 показана диаграмма контекста, которая показывает систему в окружении, выделяя основные внешние системы и пользователей, которые

взаимодействуют с ней. На рисунке 2 – диаграмма контейнеров, показывающая взаимодействие микросервисов и сторонних систем. На рисунке 3 отображена диаграмма компонентов реализованного микросервиса экспорта результатов. На 4 рисунке показана диаграмма классов и структура кода в целом.

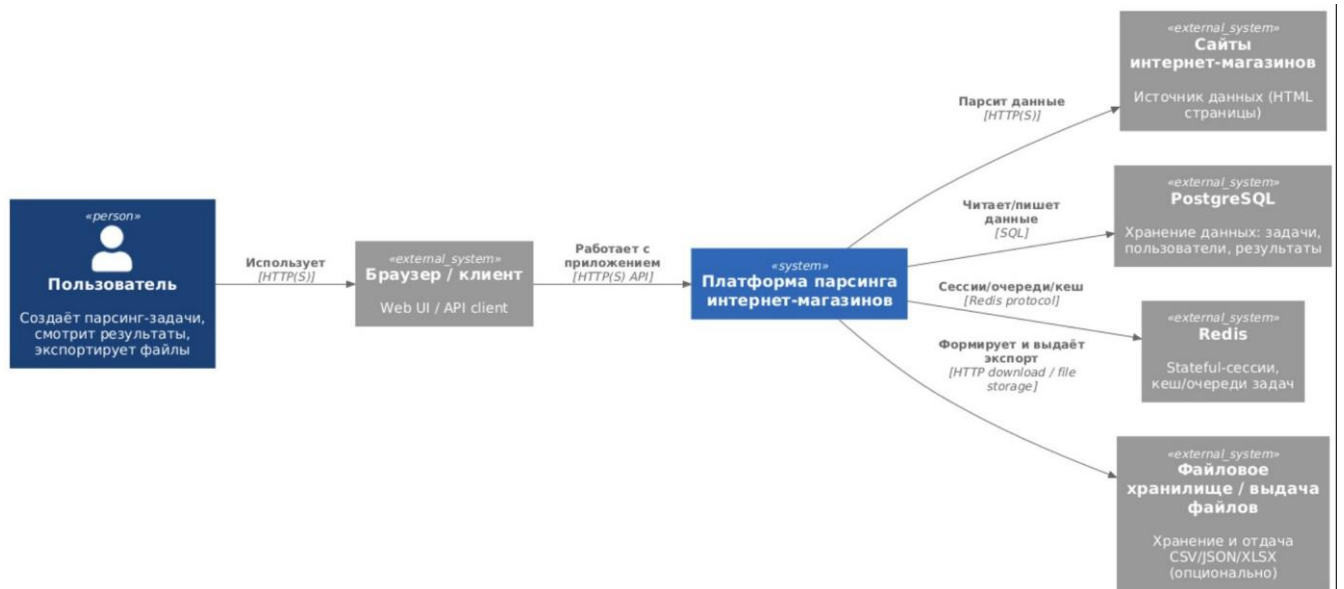


Рисунок 1 – Диаграмма контекста

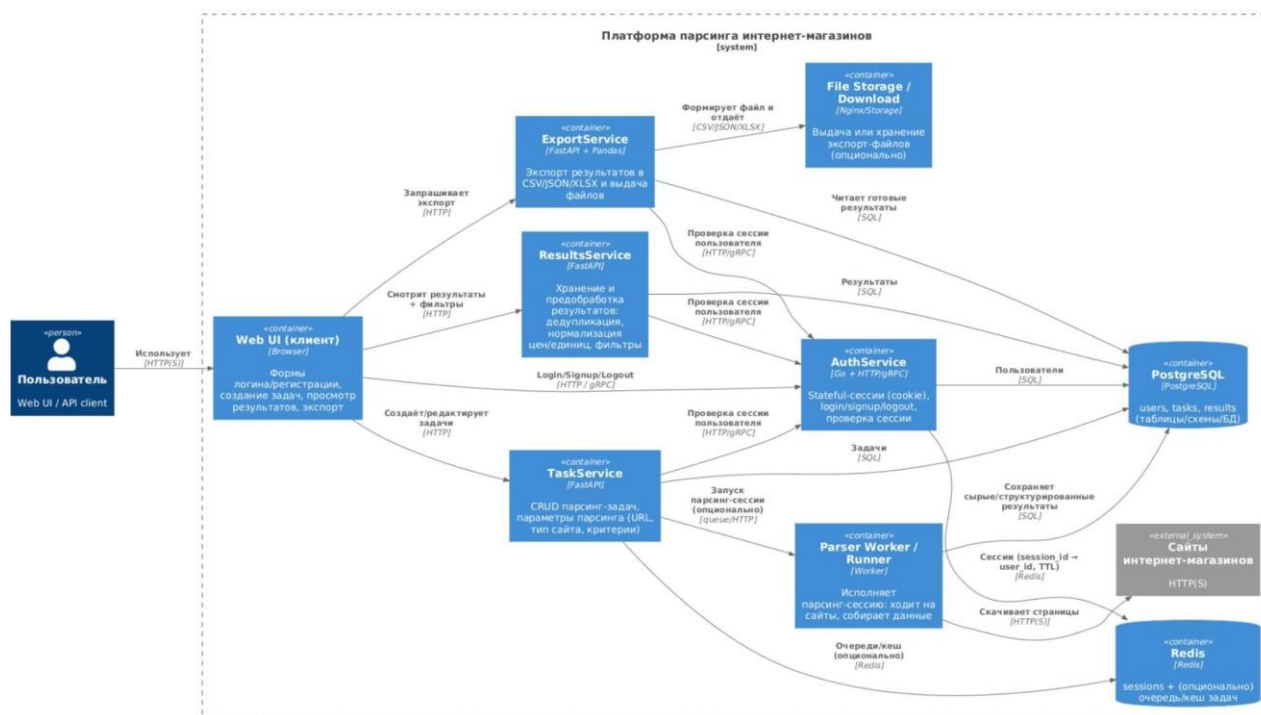
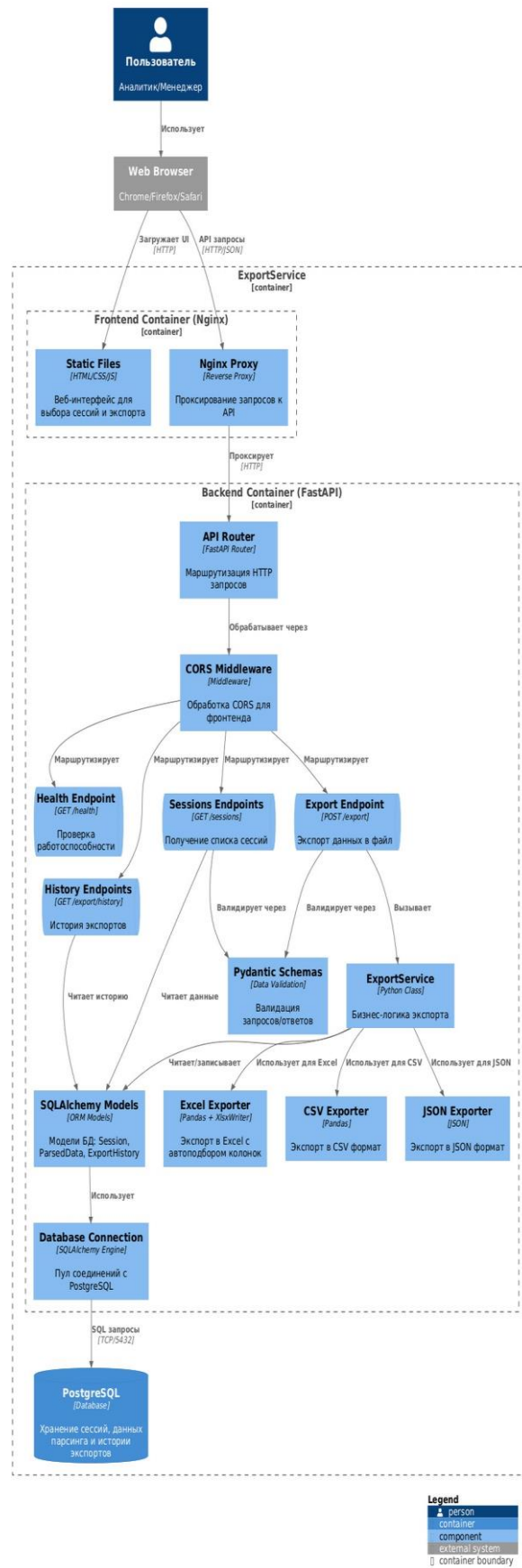


Рисунок 2 – Диаграмма контейнеров

Component Diagram (C3) - ExportService



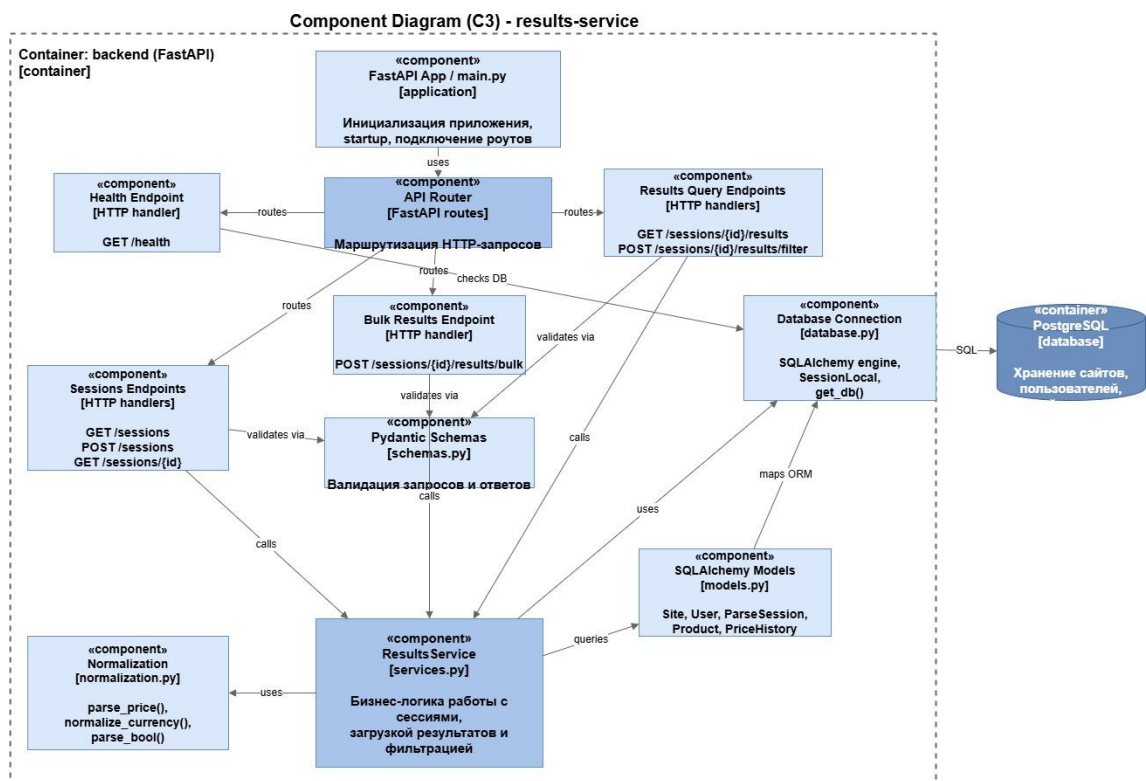


Рисунок 3 – Диаграмма компонентов

Code Diagram (C4) - ResultsService Classes

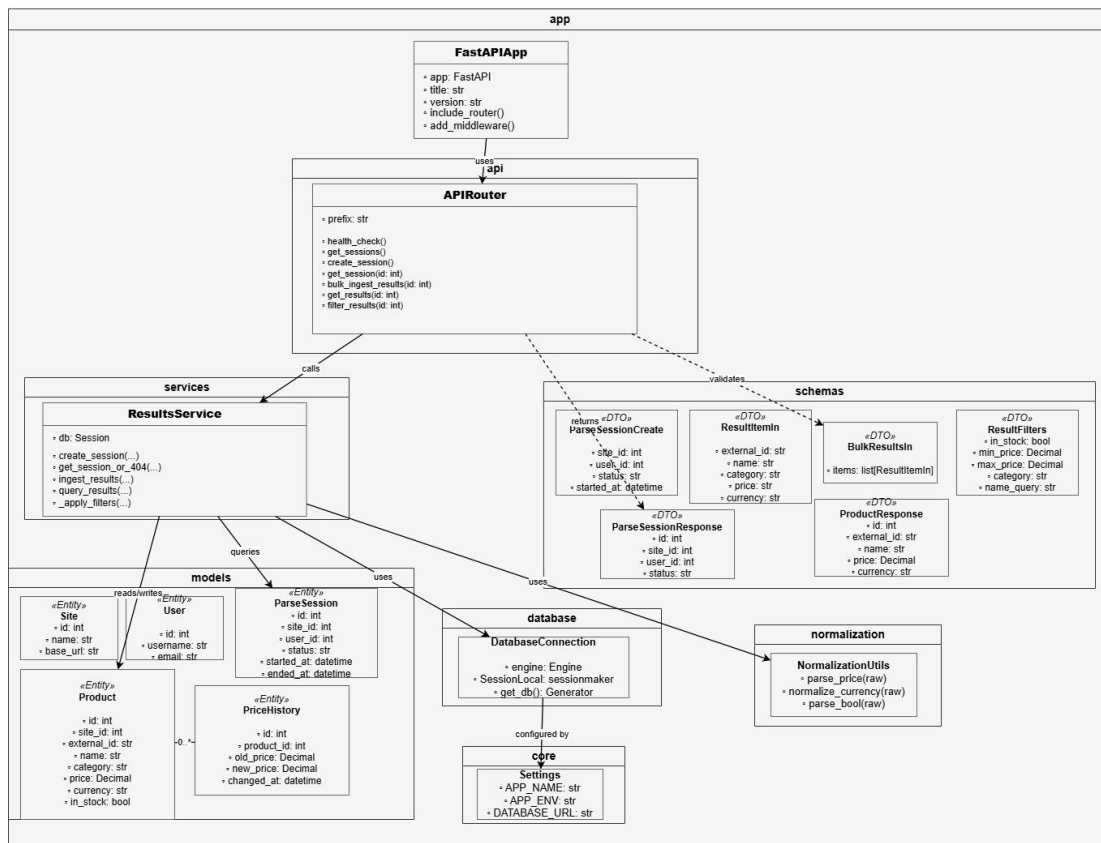


Рисунок 4 – Диаграмма кода

3 Детали реализации:

В ходе разработки были реализованы REST API endpoints, обеспечивающие основной функционал сервиса обработки результатов парсинга. Endpoint GET /api/v1/health выполняет проверку работоспособности сервиса и корректности подключения к базе данных. Endpoint GET /api/v1/sessions возвращает список всех доступных сессий парсинга с основной информацией: идентификатором, статусом и датой создания. Endpoint GET /api/v1/sessions/{id} предоставляет детальную информацию о конкретной сессии.

Создание новой сессии парсинга выполняется через endpoint POST /api/v1/sessions, который принимает параметры сессии и сохраняет их в базе данных. Для загрузки результатов парсинга реализован endpoint POST /api/v1/sessions/{id}/results, принимающий набор товаров в формате JSON и выполняющий их сохранение в базе данных. При добавлении данных выполняется нормализация значений: цена приводится к числовому формату, валюты стандартизируются, а текстовые значения обрабатываются для унификации.

Для получения результатов парсинга используется endpoint GET /api/v1/sessions/{id}/results, который возвращает список товаров, связанных с выбранной сессией. Сервис поддерживает фильтрацию данных по различным параметрам, таким как наличие товара на складе, диапазон цен, категория товара и текстовый поиск по названию. Это позволяет гибко работать с результатами парсинга и получать только необходимые данные.

Серверная часть приложения реализована на языке Python с использованием фреймворка FastAPI, который обеспечивает высокую производительность и удобную реализацию REST API. Для хранения данных используется система управления базами данных PostgreSQL, а взаимодействие с базой данных реализовано через ORM SQLAlchemy.

Вся инфраструктура приложения развёртывается в контейнерах Docker. Для управления контейнерами используется Docker Compose, который запускает сервис backend и базу данных PostgreSQL в единой изолированной сети. Такой подход обеспечивает воспроизводимость окружения, упрощает развертывание системы и позволяет легко переносить приложение между различными средами разработки и

эксплуатации.

Развертывание сервиса осуществляется с использованием Docker Compose, что позволяет запустить всю систему одной командой. При запуске создаются два контейнера: `results_backend`, содержащий серверное приложение на основе FastAPI, и `results_postgres`, в котором развёрнута база данных PostgreSQL для хранения результатов парсинга и информации о сессиях.

Контейнер `results_backend` запускает REST API сервис на порту 8000, обеспечивающий обработку HTTP-запросов, работу с базой данных и выполнение основных операций системы. Контейнер `results_postgres` работает на стандартном порту 5432 и используется для хранения данных о сессиях парсинга, товарах и истории изменений цен.

При запуске системы автоматически выполняется подключение приложения к базе данных и инициализация структуры таблиц через ORM SQLAlchemy. Все контейнеры запускаются в общей изолированной сети Docker, что обеспечивает корректное взаимодействие между сервисами и предсказуемость среды выполнения.

Вывод: был успешно спроектирован и реализован микросервис просмотра результатов парсинга.